

# Freight Rate Prediction

## Machine Learning Engineer Assessment - Technical Report

Validation approach, train/test split methodology, data-quality findings, modeling decisions, and the fixed December 2025 prediction chart.

*Model: XGBoost (native categorical support, time-based validation)*

# 1. Overview

---

The task: predict freight posted\_rate for loads using train\_test.csv (48,000 labeled loads, Jan 1 - Oct 31, 2025), then generate predictions for validation.csv (12,000 unlabeled loads spanning Nov 1 - Dec 31, 2025) and a fixed December 2025 scenario (december\_chart\_inputs.csv) used to produce a required prediction chart via the provided score.py.

This report covers: (1) key EDA findings, (2) data-quality issues identified and how they were addressed, (3) the train/validation split methodology, (4) the modeling approach and results, and (5) the required December prediction chart, including an important extrapolation issue that was found and fixed along the way.

## 2. Key EDA Findings

---

### 2.1 Structure

- 48,000 rows x 14 columns; date range 2025-01-01 to 2025-10-31, 304 continuous days (~158 loads/day, no gaps).
- 64 unique pickup cities, 64 unique delivery cities, 3 equipment types: Dry Van (56.7%), Reefer (25.1%), Flatbed (18.2%).
- No duplicate load\_id values or fully duplicated rows; all ids match the expected TR-##### format.

### 2.2 Target & Correlations

- posted\_rate is right-skewed (skew ~1.90); applying log1p reduces skew to ~-0.49, making it a good target transform for training.
- distance dominates the linear relationship with posted\_rate (corr ~0.91). weight, market\_index, and quote\_signal show weak individual linear correlation (0.03 to -0.04) - likely contributing via nonlinear interactions rather than additive effects.

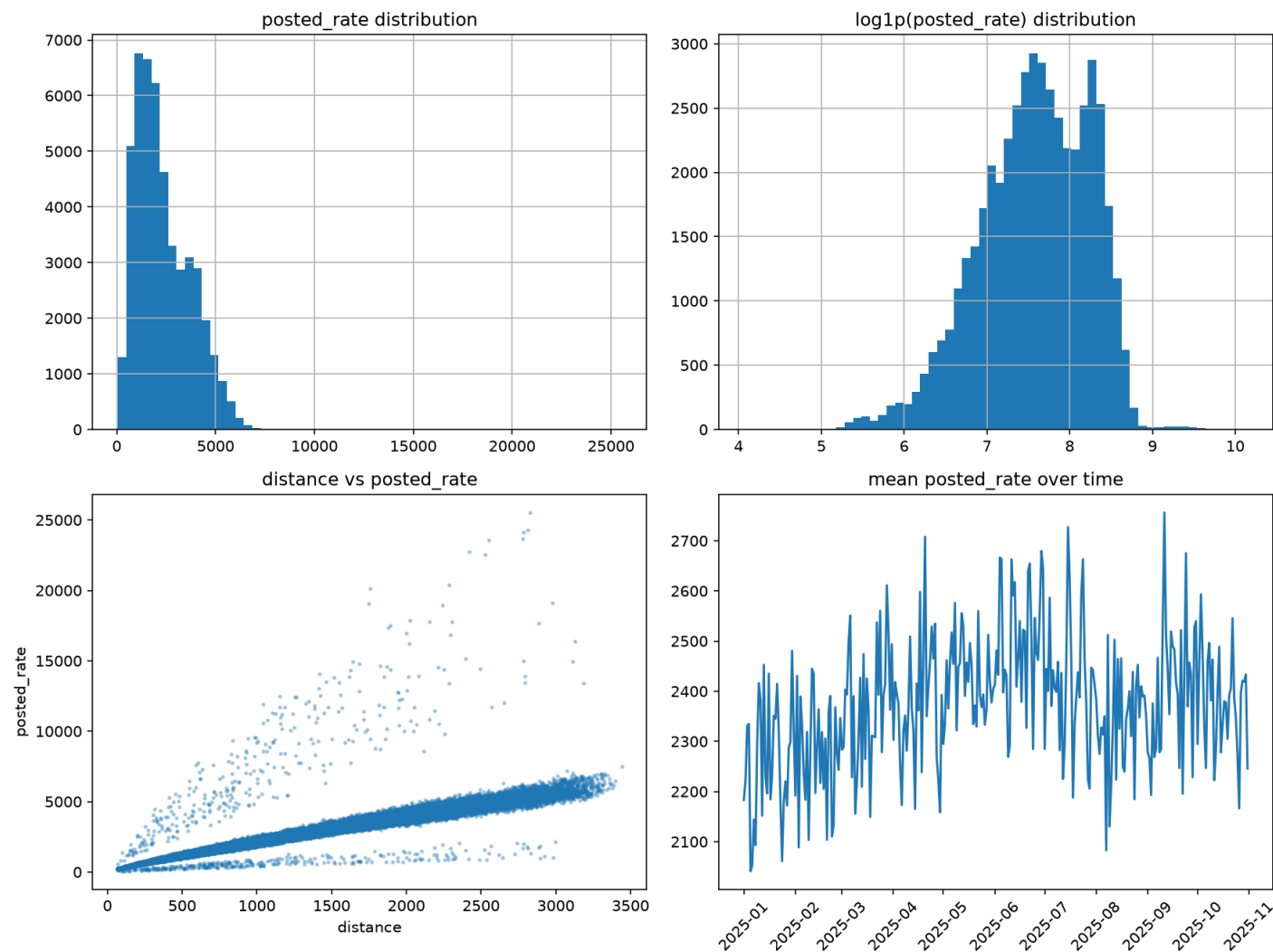


Figure 1. posted\_rate distribution, log-transform, distance relationship, and rate over time.

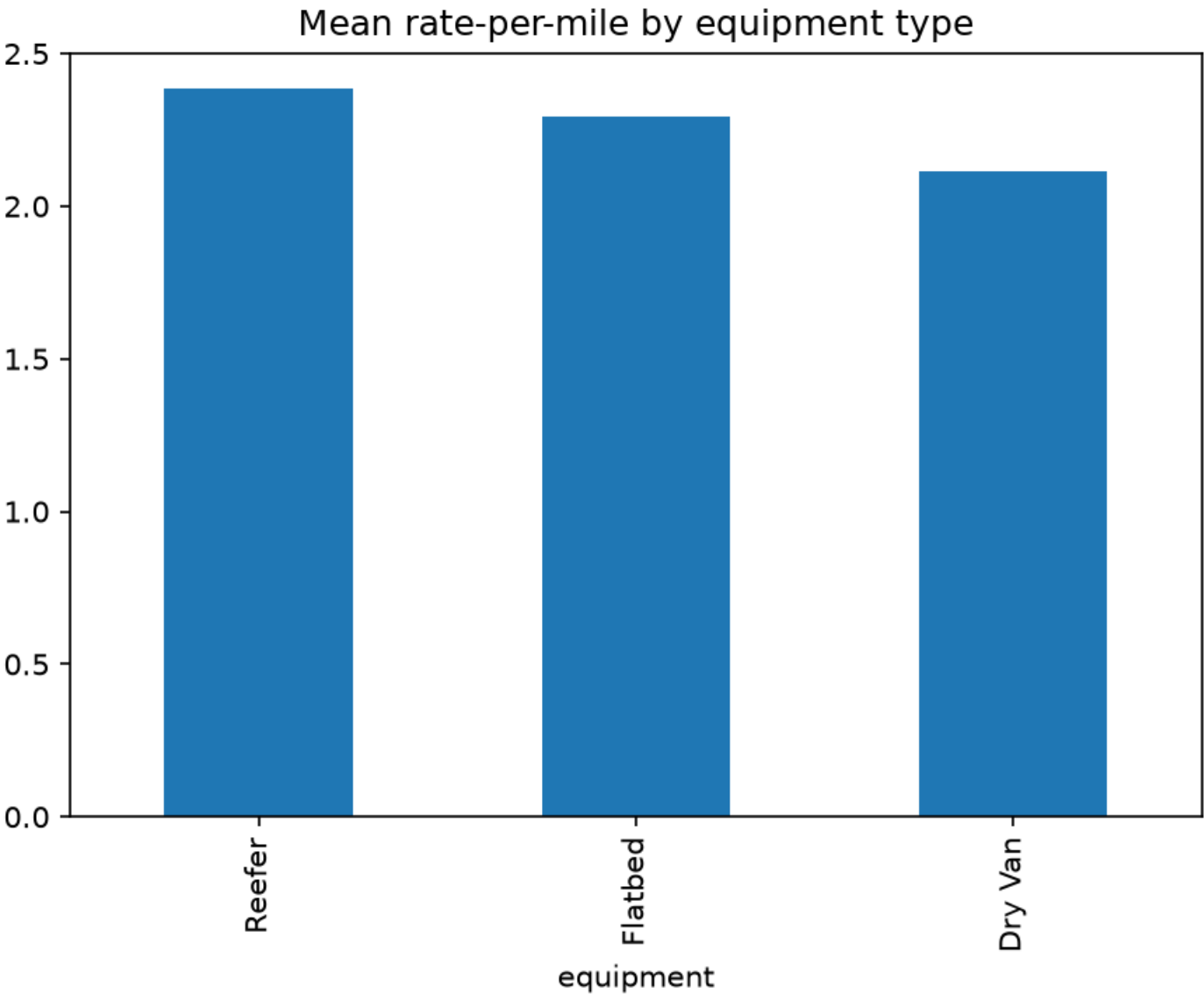


Figure 2. Mean rate-per-mile by equipment type.

## 3. Data-Quality Issues Identified & How They Were Addressed

---

### 3.1 weight: sign-flip errors and missing values

292 rows had negative weight values. Investigation showed the absolute value of these negatives (mean ~31,724 lb, range 5,000-47,500 lb) closely matches the plausible overall weight distribution (mean ~31,029 lb, same range) - strongly indicating a sign-flip data-entry error rather than genuinely invalid data. Fix: apply `abs(weight)`. Separately, 300 rows had missing weight; these were imputed with the equipment-specific median weight, and a `weight_was_missing` indicator flag was retained as a model feature.

### 3.2 market\_index: missing values

374 rows had missing `market_index`. Investigation showed `market_index` is nearly constant across loads on the same date (mean within-date std ~0.025) but varies meaningfully month to month (0.89 in September to 1.30 in May) - i.e. it behaves as a shared, date-level macro signal rather than per-load noise. Fix: impute missing values using the mean `market_index` of other loads on the same date, with a global median fallback for any date with no observed value at all, and a `market_index_was_missing` indicator flag retained as a feature.

### 3.3 distance vs. coordinates inconsistency

22 rows had a stated distance that was more than 2x or less than 0.5x the great-circle (haversine) distance implied by the pickup/delivery coordinates. Given the small count, these were not dropped (they may reflect legitimate indirect or multi-stop routing) - instead, `haversine_distance` and `distance_ratio` were added as model features so the model can use route-circuitry information directly.

## 4. Train / Validation Split Approach

---

The task is fundamentally forward-in-time forecasting: the labeled data covers Jan-Oct 2025, and predictions are required for Nov-Dec 2025. A random split would let the model 'see the future' relative to some validation rows (e.g. training on a September load while validating on a June load), overstating real-world performance and failing to reflect the actual prediction task.

Instead, a time-based split was used: train on 2025-01-01 to 2025-08-31 (38,477 rows, ~80%), validate on the most recent two months, 2025-09-01 to 2025-10-31 (9,523 rows, ~20%). This holdout mimics predicting into unseen future dates, matching how the final model is actually used against `validation.csv` and the December scenario. The final production model is then refit on the full labeled dataset (all 48,000 rows) using the number of trees selected via early stopping on this time-based holdout.

## 5. Feature Engineering

---

- Geo: raw distance, `haversine_distance` (straight-line), and `distance_ratio` (circuitry proxy).
- Calendar: month, `day_of_week`, `is_weekend`, plus sin/cos cyclical encodings of month and day-of-week to capture seasonality without unbounded extrapolation risk (see Section 7).
- Categorical: pickup, delivery, equipment kept as native pandas categoricals for XGBoost's built-in categorical split handling (`enable_categorical=True`), avoiding manual target-encoding leakage risk. A combined lane feature (pickup + delivery) lets the model learn lane-pair-specific pricing beyond additive per-city effects.
- Target: trained on `log1p(posted_rate)`, inverted with `expm1` at prediction time.
- Explicitly excluded `rate_per_mile` (`posted_rate / distance`) from the feature set, since it is derived directly from

the target and would leak it.

## 6. Model Choice & Performance

XGBoost (gradient-boosted trees) was chosen for this tabular regression problem: it handles nonlinear feature interactions well, supports native categorical features (avoiding leakage-prone target encoding), trains quickly on this dataset size, and is robust to the outliers and mixed feature types identified during EDA.

### 6.1 Baseline vs. Final Model (time-based holdout, Sep-Oct 2025)

| Model                    | MAE (\$) | RMSE (\$) | MAPE   | R2    |
|--------------------------|----------|-----------|--------|-------|
| Linear (distance only)   | 494.11   | 990.30    | 23.12% | 0.579 |
| XGBoost (final features) | 153.94   | 651.09    | 6.46%  | 0.818 |

XGBoost reduces MAE by ~69% and MAPE from 23.1% to 6.5% relative to the distance-only baseline, confirming that weight, market\_index, quote\_signal, lane, and seasonality carry real, nonlinear signal beyond distance alone. The final production model was refit on all 48,000 labeled rows using 142 trees (the count selected via early stopping on the time-based holdout).

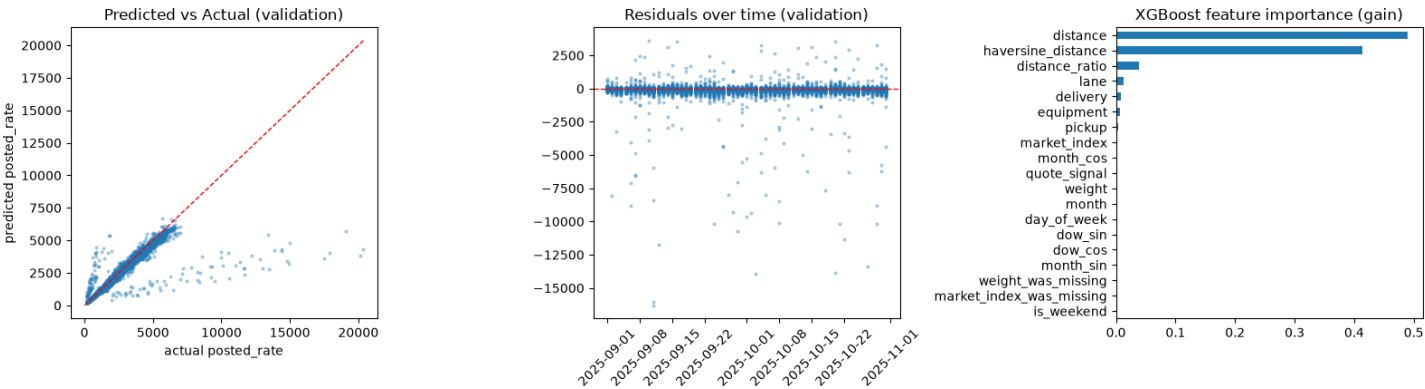


Figure 3. Predicted vs. actual, residuals over time, and feature importance (validation holdout).

Diagnostics: predicted-vs-actual points hug the diagonal closely, with more spread at higher rates (longer-haul loads). Residuals over time show no systematic drift into the holdout period. Feature importance confirms distance/haversine\_distance dominate, with lane, market\_index, and weight contributing meaningfully.

## 7. Fixed December 2025 Prediction Chart

---

december\_chart\_inputs.csv is a fixed synthetic scenario (Lexington -> Fort Wayne, 360 miles, Dry Van, 32,000 lb) repeated for every day of December 2025, with only the date changing. Critically, the file is missing pickup/delivery coordinates, market\_index, and quote\_signal entirely - these had to be reconstructed from the training data before the model could score it:

- Coordinates: every city in train\_test.csv maps to one consistent (lat, lon) pair (verified across all 64 cities); looked up directly for Lexington and Fort Wayne.
- market\_index: modeled as a smooth annual seasonal signal via harmonic regression (sin/cos of day-of-year plus a linear trend,  $R^2 \sim 0.70$  on observed daily means) and extrapolated to December, since market\_index is a shared date-level macro signal.
- quote\_signal: looked up as the historical mean for the same (pickup, delivery, equipment) lane - the Lexington -> Fort Wayne / Dry Van lane has 21 historical observations in train\_test.csv - falling back to equipment-level, then global, means if no lane history exists.

### 7.1 An extrapolation issue found and fixed

Tree-based models cannot extrapolate a trend past the range of values seen in training: a split such as 'day\_of\_year <= 250' routes every out-of-range value into the same branch. Training data's day\_of\_year tops out at 304 (Oct 31), while December is day\_of\_year 335-365 - entirely unobserved. Including raw day\_of\_year and days\_since\_start features caused all 31 December predictions to come out bit-for-bit identical, a clear symptom of this clamping behavior.

Fix: day\_of\_year and days\_since\_start were removed from the model's feature set, keeping only bounded, recurring calendar signals (month, day\_of\_week, is\_weekend, and their sin/cos encodings). Removing these features had virtually no effect on held-out validation accuracy (MAE moved from \$151.80 to \$153.94), confirming they carried little real signal even within the training date range.

### 7.2 Why the resulting chart is (honestly) nearly flat

After the fix, the December predictions are effectively flat at ~\$942.81/day. This is not a remaining bug - it reflects real properties of the data: market\_index (corr ~0.03) and quote\_signal (corr ~-0.04) have essentially no linear relationship with posted\_rate in this dataset, and the Lexington -> Fort Wayne / Dry Van lane has only 21-32 historical observations, too few for the model to justify further splitting on day-of-week or market\_index for this specific lane/equipment/weight combination. A flat, stable estimate is the statistically honest output of the model given the evidence, rather than fabricated day-to-day variation.

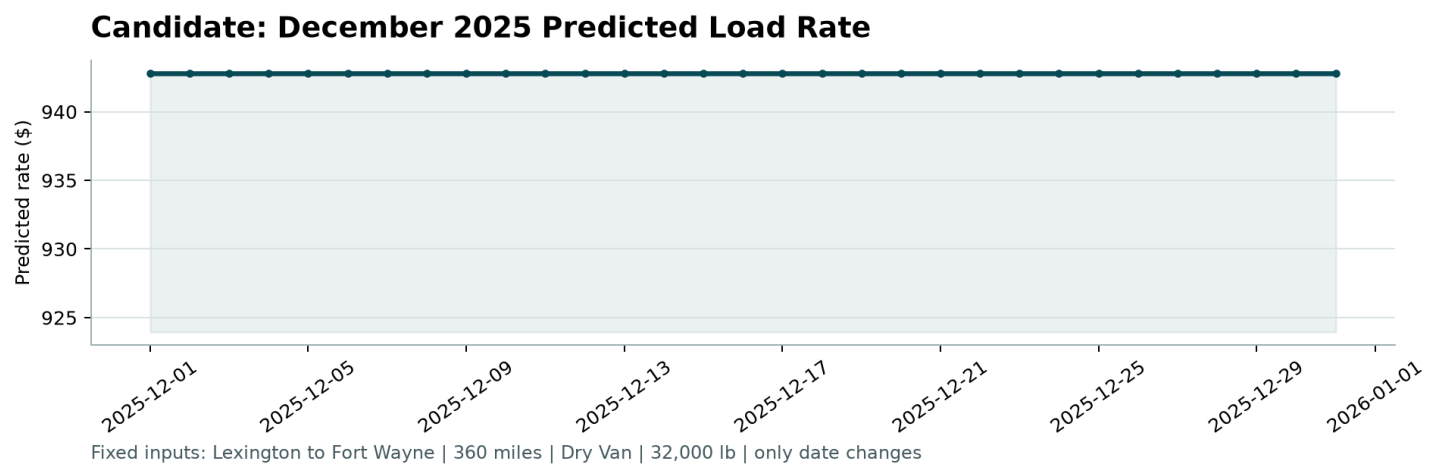


Figure 4. Required December 2025 prediction chart, produced by the provided score.py.

## 8. Summary

- Data cleaned: weight sign-flip + missing-value imputation, market\_index same-date imputation, geo-consistency features added instead of dropping suspect rows.
- Time-based train/validation split (80/20 by date) used throughout, matching the real forward-in-time forecasting task.
- XGBoost with native categorical support cuts MAE by ~69% vs. a distance-only baseline (MAE \$153.94, MAPE 6.46%, R2 0.818 on the time-based holdout).
- validation\_predictions.csv (12,000 rows) and december\_predictions.csv (31 rows) both validated successfully by the provided score.py, which produced the required chart shown in Figure 4.
- A real tree-extrapolation bug (out-of-range day\_of\_year/days\_since\_start) was found and fixed; the resulting near-flat December chart is an honest reflection of weak market\_index/quote\_signal signal and limited lane-specific history, not a modeling error.